

به نام خدا



درس سیستم عامل
نیم سال اول ۰۵-۰۴
استاد: دکتر اسدی

دانشکده مهندسی کامپیوتر

تمرین سری پنجم

- پرسش‌های خود را در سامانه CW و تالار مربوط به تمرین مطرح نمایید.
- پاسخ سوالات را تایپ نمایید.
- اسکرین‌شات‌ها، عکس‌ها، فایل‌های مربوط به سوال عملی، گزارش تمرینات عملی و PDF قسمت تئوری را در پوشه با نام به فرمت HWNUM_StudentID1_StudentID2 ذخیره نمایید. سپس آن را zip نمایید و در صفحه درس بارگذاری نمایید. به عنوان مثال یک فایل بارگذاری شده قابل قبول باید دارای فرمت HW1_400123456_403123456.zip باشد.
- هر دانشجو می‌تواند حداکثر سه تمرین را با دو روز تأخیر بدون کاهش نمره ارسال نماید.
- تمرینات عملی و تئوری به صورت گروه‌های دو نفر تحویل داده شود.
- هر دو عضو گروه موظف هستند تمرینات خود را بارگذاری کنند.
- عواقب عدم تطابق بین پاسخ دو عضو گروه برعهده خودشان است.
- تحویل تمرین به صورت انگلیسی مجاز نیست. در صورت تحویل تمرین به صورت انگلیسی (حتی بخشی از تمرین) نمره تمرین موردنظر صفر در نظر گرفته می‌شود.
- در صورت مشاهده تقلب برای بار اول نمره هر دو طرف صفر می‌شود. در صورت تکرار نمره کل تمرینات صفر خواهد شد.
- استفاده از ابزارهایی مانند ChatGPT به منظور ابزار کمک آموزشی مجاز است به شرط آن که به خروجی آن اکتفا نشود.
- توجه شود که پروژه نهایی درس در گروه‌های چهار نفر تحویل گرفته می‌شود.
- سوالات با عنوان اختیاری نمره‌ای ندارند اما جواب دادن به آن‌ها کمک به سزایی در یادگیری درس می‌کند.
- هیچ تمرینی خارج از CW تصحیح نخواهد شد. لذا توصیه می‌شود حداقل یک الی دو ساعت قبل از زمان پایان تمرین برای بارگذاری تمرین اقدام شود.

تمارین تئوری

۱. یک CPU از یک زمانبند سه سطحی Multilevel Feedback Queue (MLFQ) استفاده می‌کند. ویژگی‌های این زمانبند در جدول زیر آمده است:

صف	سیاست زمانبندی	کوانتوم زمانی (ms)
Q ₀	RR	۴
P _۲	RR	۸
P _۳	FCFS	–

- در Q₀ قاعده تنزل به این شکل است که اگر پردازش دو کوانتوم پی در پی در Q₀ اجرا شود و تمام نشود به Q₁ تنزل می‌کند.
- در Q₁ قاعده تنزل به این شکل است که اگر پردازش یک کوانتوم پی در پی در Q₁ اجرا شود و تمام نشود به Q₂ تنزل می‌کند.
- اگر یک پردازش به مدت ۲۰ میلی‌ثانیه یا بیشتر در Q₂ بماند؛ به علت سالخورده‌گی^۱ Q₁ ارتقا می‌یابد.

فرضیات:

- ترتیب اولویت صف‌ها به صورت $Q_0 > Q_1 > Q_2$ است (Q₀ بالاترین اولویت را دارد).
- زمانبند همواره پردازش‌های را از بالاترین صف غیرخالی انتخاب می‌کند.
- اگر یک پردازش پیش از اتمام کوانتوم خود پایان یابد، تنزل نمی‌یابد.
- ارتقاء یا تنزل صف در مرز کوانتوم و بلافاصله اعمال می‌شود.
- سربار تغییر متن^۲ قابل چشم‌پوشی است.

چهار پردازش زیر به سیستم وارد می‌شوند:

پردازش	زمان ورود (ms)	burst (ms)	صف اولیه
P _۱	۰	۲۲	Q _۰
P _۲	۱	۷	Q _۰
P _۳	۳	۱۵	Q _۱
P _۴	۵	۱۲	Q _۲

آ) اجرای تمام پردازش‌ها را شبیه‌سازی کنید و نمودار گانت مربوطه را رسم کنید تا نشان دهد در هر بازه زمانی کدام پردازش در حال اجرا است.

ب) لحظاتی را که پردازش‌ها دچار ارتقاء^۳ یا تنزل^۴ می‌شوند، مشخص کنید.

ج) برای هر پردازش و برای کل سیستم، موارد زیر را محاسبه کنید:

• Turnaround time = زمان اتمام – زمان ورود

• Waiting time = turnaround – burst

• میانگین turnaround time و میانگین waiting time

د) توضیح دهید که قاعده سالخورده‌گی در Q_۲ چگونه از گرسنگی^۵ جلوگیری می‌کند.

¹ Aging

² Context Switch Overhead

³ Promotion

⁴ Demotion

⁵ Starvation

۲. قطعه کد زیر که بخشی از پیاده‌سازی منطق یک زمان‌بند است را در نظر بگیرید:

```

1 // Each Thread has base_prio (base priority) and cur_prio (current priority
2 // Smaller number -> higher priority.
3 typedef struct {
4     char *id;
5     int base_prio, cur_prio;
6     bool holds_lock;
7 } Thread;
8 typedef struct {
9     Thread *owner;
10    Queue waiters;
11 } Lock;
12
13 void lock_acquire(Lock *L, Thread *T) {
14     if (L->owner == NULL) {
15         L->owner = T; T->holds_lock = true;
16     } else {
17         enqueue(&L->waiters, T);
18         // Priority inheritance: lock owner should inherit the highest
19         // waiting priority.
20         if (L->owner->cur_prio < T->cur_prio)
21             L->owner->cur_prio = T->cur_prio;
22     }
23 }
24
25 void lock_release(Lock *L) {
26     // When releasing, restore owner's priority to its base value.
27     L->owner->cur_prio = L->owner->base_prio;
28     L->owner->holds_lock = false;
29     if (!empty(L->waiters))
30         L->owner = dequeue(&L->waiters);
31     else
32         L->owner = NULL;
33 }

```

آ) خطای منطقی که در پیاده‌سازی این قطعه کد وجود دارد را شرح دهید.

ب) یک مثال عددی (سه ریسمان) با زمان‌بندی و نمودار زمانی ارائه دهید که نشان دهد این خطا چگونه باعث Priority Inversion یا کندی یک ریسمان با اولویت بالا می‌شود.

ج) قطعه کد را به نحوی تغییر دهید که خطای مورد نظر برطرف شود (راهنمایی: تنها یک خط نیاز به تغییر دارد).

د) در پیاده‌سازی تابع lock_release نکته‌ای در ارتباط با بازگشت اولویت پس از آزادسازی قفل رعایت نشده است. این نکته را بیابید و آن را شرح دهید.

۳. به سوالات هر بخش پاسخ دهید:

الف) درستی یا نادرستی موارد زیر را با ذکر دلیل مشخص کنید.

آ) زمان‌بندی Round Robin هنگامی که q بسیار کوچک است، مانند زمان‌بندی First Come First Serve عمل می‌کند.

ب) اگر یک پردازش از زمان اختصاص یافته‌اش استفاده کند، یک وقفه‌ی تایمر^۶ رخ می‌دهد و پردازش در صف Blocked قرار می‌گیرد.

^۶Timer Interrupt

- (ج) یک سیستم عامل از یک تایمر ساعت برای اتخاذ تصمیم‌های زمان‌بندی پردازش‌ها استفاده می‌کند. افزایش فرکانس وقفه‌ی این ساعت، مدت زمان اجرای برنامه‌ی شما را کاهش می‌دهد.
- (ب) برای هر یک از موارد زیر، از چه سیاست زمان‌بندی استفاده می‌کنید؟ دلایل خود را توضیح دهید:
- (آ) پردازش‌ها در فواصل زمانی نسبتاً بزرگ وارد می‌شوند.
- (ب) کارایی سامانه با درصد کارهای تکمیل‌شده^۷ سنجیده می‌شود.
- (ج) همه پردازش‌ها تقریباً به یک اندازه زمان برای اجرا نیاز دارند.
- (ج) در نظر بگیرید یک سیستم تک‌پردازنده که فقط وظیفه‌های CPU-bound دارد و زمان‌بندی همیشه غیرقابل قطع^۸ انجام می‌شود. فرض کنید زمان اجرا هر وظیفه از قبل معلوم است، هیچ دو وظیفه‌ای زمان اجرای برابر ندارند، و همه‌ی وظیفه‌ها از ابتدای سیستم حاضر هستند. نشان دهید که زمان‌بند Shortest-Job-First (SJF) تنها ترتیب بهینه از نظر میانگین زمان بازگشت^۹ است، که در آن زمان بازگشت به صورت مدت زمان از ورود وظیفه به سیستم تا پایان آن تعریف می‌شود.

۴. به سوالات زیر پاسخ دهید:

- (آ) تحت چه شرایطی سیاست زمان‌بندی Priority-Based می‌تواند مشابه با سیاست SJF عمل کند؟ با ذکر یک مثال عملی توضیح دهید.
- (ب) با توجه به تفاوت در زمان ورود پردازش‌ها و تعداد پردازش‌های جدید وارد شده در سیستم، بیان کنید که زمان انتظار پردازش‌ها چگونه تحت تأثیر سیاست Round Robin با کوانتوم زمانی متغیر قرار می‌گیرند. برای هر حالت (افزایش یا کاهش کوانتوم زمانی) توضیح دهید که چگونه زمان انتظار تغییر می‌کند.
- (ج) برای یک سیستم چند پردازنده‌ای که از الگوریتم زمان‌بندی Multilevel Feedback Queue استفاده می‌کند، توضیح دهید که چگونه فرآیندهای I/O-bound از نظر زمان‌بندی با فرآیندهای CPU-bound متفاوت هستند. نقش سطح‌های مختلف در بهینه‌سازی اجرای این دو نوع فرآیند را توضیح دهید.
- (د) در یک سیستم بی‌درنگ که از الگوریتم EDF استفاده می‌کند، فرض کنید که دو پردازش با زمان اجرای متفاوت و ضرب‌الاجل‌های مختلف وارد صف شوند. چگونه می‌توان تعیین کرد که آیا سیستم می‌تواند به تمام ضرب‌الاجل‌ها دست یابد؟ چه مولفه‌هایی در این تصمیم‌گیری مهم هستند؟
- (ه) در سیستم‌های بی‌درنگ، هنگامی که کارها را بر اساس اولویت زمان‌بندی می‌کنیم و همچنین سازوکارهای انحصار متقابل در دسترسی به منابع را نیز در اختیار داریم، پدیده‌ای به نام Priority Inversion یا وارونگی اولویت می‌تواند رخ دهد. در خصوص این پدیده تحقیق کنید و آن را با یک مثال توضیح دهید. سپس در خصوص روش اثربری اولویت^{۱۰} که در حل کردن این مورد موثر است تحقیق کنید و آن را توضیح دهید.

۵. به سوالات زیر پاسخ دهید:

- (آ) بعضی الگوریتم‌های زمان‌بندی دارای مولفه‌هایی هستند، مثلاً اندازه کوانتوم که مولفه‌ی الگوریتم RR است. بنابراین می‌توان به این الگوریتم‌ها به صورت مجموعه‌ای از زمان‌بندها نگاه کرد، مثلاً الگوریتم RR به ازای مقادیر کوانتوم مختلف، یک مجموعه نامتناهی از زمان‌بندها را ارائه می‌کند. این مجموعه‌ها می‌توانند شامل سایر الگوریتم‌ها باشند، مثلاً الگوریتم FCFS معادل الگوریتم RR با کوانتوم بی‌نهایت است. با توجه به این موضوع رابطه بین جفت الگوریتم‌های زیر را توصیف کنید.

(آ) Priority and SJF

^۷Completed Jobs

^۸Non-Preemptive

^۹Turnaround Time

^{۱۰}Priority Inheritance

(ب) Multilevel feedback queues and FCFS

(ج) Priority and FCFS

(د) RR and SJF

(ه) RM and Priority

(ب) چرا الگوریتم EDF نسبت به سایر الگوریتم‌ها در شرایط بی‌درنگ برتری دارد؟ درستی یا نادرستی هر مورد را با ذکر دلیل بیان نمایید.

(آ) این الگوریتم براساس ضرب الاجل‌ها تصمیم‌گیری می‌کند و در نتیجه می‌تواند تضمین‌هایی برای شرایط بی‌درنگ ارائه دهد که سایر الگوریتم‌ها نمی‌توانند این کار را انجام دهند.

(ب) این الگوریتم Preemption ندارد و در نتیجه پیاده‌سازی آن ساده‌تر است.

(ج) این الگوریتم بیش‌ترین پاسخگویی ممکن را، حتی در حضور وظایف طولانی مدت محاسباتی ارائه می‌دهد.

(د) از آنجایی که وظایف براساس ضرب الاجل‌ها مرتب می‌شوند، این الگوریتم در شرایطی که پردازنده Over-Subscribe شده است، بهتر عمل می‌کند.

۶. با توجه به جدول فرآیندهای زیر که زمان ورود و مدت زمان CPU مورد نیاز برای هرکدام از آن‌ها را نشان می‌دهد، زمان‌بندی‌های خواسته شده را بررسی کنید. نمودار گانت و جدول زمان‌بندی را رسم کنید.

(آ) FCFS

(ب) SRTF

(ج) RR با $\text{quantum} = 3$

(د) RR با $\text{quantum} = 1$

Proc Id	Arrival Time	Computation Time
A	0	2
B	1	6
C	4	1
D	7	4
E	8	3

تمارین عملی

۱. XV6 OS

برای این تمرین از کدی که در سامانه CW قرار گرفته است استفاده کنید!

در این تمرین قصد داریم تا الگوریتم Lottery Scheduling را برای سیستم عامل xv6 پیاده سازی کنیم. Lottery Scheduling یک الگوریتم احتمالی است که برای زمان بندی CPU استفاده می شود. در این الگوریتم به هر پردازش تعدادی بلیت لاتاری یا همان ticket اختصاص داده می شود که نشان دهنده سهم این پردازش در استفاده از CPU است. پس از اختصاص این بلیت ها، پردازنده یک قرعه کشی تصادفی را انجام می دهد تا پردازش را برای اجرا انتخاب کند. هر پردازشی که تعداد بلیت بیشتری داشته باشد، شانس انتخاب شدن آن نیز بیشتر است. برای پیاده سازی این الگوریتم، مراحل زیر را باید به ترتیب انجام دهید. دقت کنید که کدهای تغییر یافته و اضافه شده خود را به همراه توضیح مختصری در سامانه بارگذاری کنید.

۱. در مرحله اول باید یک متغیر با عنوان ticket در پردازش های خود تعریف کنید (می توانید struct proc را تغییر دهید).

۲. در ادامه باید هنگام ساخت یک پردازش جدید، یک مقدار اولیه به آن بدهید. همچنین باید fork را به گونه ای تغییر دهید که یک پردازش فرزند، تعداد ticket را از پدر خود به ارث ببرد.

۳. حال نیاز داریم تا تابع schedule را در kernel/proc.c به گونه ای تغییر دهیم تا الگوریتم زمان بندی جدید را پشتیبانی کند. برای این کار نیاز است که هر دفعه تعداد تمامی ticket هایی که در همه پردازش های RUNNABLE وجود دارند را محاسبه کنید و سپس با کمک گرفتن از تابع تصادفی که برای شما با نام rand_int قرار گرفته است، یک پردازش را بر اساس ticket های انتخاب کنید و آن را اجرا کنید. دقت کنید که هر پردازشی که تعداد ticket های بیشتری داشته باشد، شانس بیشتری برای اجرا دارد.

۴. در ادامه شما نیاز دارید تا یک syscall اضافه کنید تا بتواند تعداد ticket یک پردازش خاص را تغییر دهد. به این منظور این syscall باید شماره pid و تعداد ticket را به عنوان ورودی بگیرد و سپس آن تعداد ticket را به آن پردازش اختصاص دهد. دقت کنید که اگر PID مورد نظر وجود نداشت یا تعداد ticket مقدار معتبری نبود، باید این syscall مقدار ۱- را برگرداند و نشان دهد که خطایی رخ داده است.

۵. در انتها شما نیاز دارید تا یک برنامه کاربر بنویسید که با اجرای آن، چهار پردازش جدید را fork کند و مقدار ticket آن ها را به نسبتی مشخص، با استفاده از syscall تعیین کنید. در ادامه این پردازش ها هر کدام در یک حلقه، یک متغیر مخصوص را افزایش دهند و پس از مدت کوتاهی، مقدار متغیرهای آن ها را نسبت به مقدار ticket آن ها مقایسه کنید تا از عملکرد صحیح زمان بند خود مطمئن شوید.

پیشنهاد می شود تا سیستم عامل خود را برای تست کردن در حالت تک هسته اجرا کنید. برای این کار باید دستور زیر را اجرا کنید:

```
1 CPUS=1 make qemu
```

همچنین برای اشکال زدایی (دیباگ) سیستم باید از دستور زیر استفاده کنید:

```
1 CPUS=1 make qemu-gdb
```

همچنین برای تولید یک مقدار تصادفی، تنها لازم است تا تابع rand_int را صدا بزنید و این تابع یک مقدار تصادفی ۳۲ بیتی را برمی گرداند.

۲. Docker

در این تمرین قصد داریم ایزوله‌سازی کانتینرها را از منظری متفاوت بررسی کنیم. در حال حاضر اگر دستور زیر را اجرا کنید، به سادگی به فایل‌های روی میزبان از داخل کانتینر دسترسی خواهید داشت. این در حالی است که این امکان هنگام استفاده از میزبان وجود ندارد.

```
1 ./zocker run --name hello 'sh'
```

گام‌های تمرین

بخش اول

آ. یک کانتینر ساده مانند busybox را با دستور sleep اجرا کنید.

ب. با استفاده از دستور زیر یک پوسته در کانتینر اجرا کنید و به مسیر ریشه بروید.

```
1 docker exec -it <container-id> sh
```

سوال: با توجه به درک خود از پیاده‌سازی پروژه تا اینجا، به نظرتان چگونه ممکن است، مسیر ریشه کانتینر داکر از مسیر ریشه میزبان متفاوت باشد؟ (در پاسخ به این سوال درست بودن جواب اهمیت ندارد و صرفاً حدس خود را به صورت کوتاه بیان کنید)

ج. دستور زیر را اجرا کنید و به مسیر بدست آمده بروید. (توصیه می‌شود با کاربر root وارد این مسیر شوید).

```
1 docker inspect <container-id> | jq '.[].GraphDriver.Data.MergedDir'
```

محتوای موجود در این مسیر را با ریشه کانتینر مقایسه کنید.

بخش دوم

پیش از شروع این بخش، از وجود پوشه در مسیر `/tmp/zocker` اطمینان حاصل کنید. همچنین پس از هر بار اجرای پروژه یک مرتبه دستور زیر را اجرا کنید.

```
1 rm -rf /tmp/zocker/test-container
```

همچنین آخرین نسخه مخزن را دریافت کنید و به تگ t3 بروید.

```
1 git checkout t3
```

د. با استفاده از فراخوان سیستمی (2) chroot مسیر ریشه کانتینری که اجرا می‌کنید را به `/tmp/zocker/container/test-` container تغییر دهید، به طوری که با اجرای دستور زیر دیگر دسترسی به فایل‌های روی میزبان از داخل کانتینر امکان‌پذیر نباشد.

```
1 ./zocker run --name test-container 'sh'
```

(راهنمایی: لازم است این فراخوانی در فایل `src/run.c` پس از خصوصی‌سازی ریشه انجام شود)

ه. در صورتی که به درستی بخش قبل را پیش برده باشید، باید با خطای زیر مواجه شوید.

```
1 [ERR] Failed to remount /proc: No such file or directory
```

سوال: علت بروز این خطا چیست؟

این خطا را با ساخت پوشه `/proc` پس از تغییر ریشه کانتینر اصلاح کنید. توجه داشته باشید که قبل از ساخت این پوشه، لازم است با استفاده از فراخوان سیستمی (2) chdir به مسیر ریشه جدید بروید.

و. در صورتی که بخش قبل را به درستی پیش برده باشید، پس از اجرای دوباره باید خطای زیر را مشاهده کنید:

```
1 [ERR] Failed to call create container process: No such file or directory
```

سوال: چرا این خطا رخ می‌دهد؟

با تغییر توابع `setup_bin_dir` و `setup_lib_dir` در فایل `src/setup.c` این خطا را اصلاح کنید. (راهنمایی: برای بدست آوردن لیست کتابخانه‌های مشترک برای یک فایل باینری می‌توانید از دستور `ldd` استفاده کنید) ز. با استفاده از دستورات `ls`، `cd` و `pwd` نشان دهید که ریشه کانتینر جدا شده است. ح. اختیاری: راهی پیدا کنید که از داخل کانتینر بتوانید همچنان به فایل‌های میزبان دسترسی پیدا کنید.

موارد لازم جهت تحویل

- پاسخ به سه سوال.
- نشان دادن عملکرد مورد انتظار در بخش ز.
- کدهای تغییر یافته.

۳. توسعه سیستم مدیریت پرونده

این تمرین در ادامه تمرین قبل طرح شده است و قصد داریم سیستم مدیریت پرونده خود را تکمیل کنیم.

گام‌های تمرین

۱. یک رابط کاربری `cli` برای دستورات تمرین گذشته آماده کنید و عملکرد آن را آزمایش کنید.
۲. در این تمرین قصد داریم فضای خالی را به شکل بهتری مدیریت کنیم. برای این منظور داده ساختار `freelist` را به یک لیست پیوندی تغییر دهید به طوری که اعضای لیست پیوندی ساختارهایی شامل شروع بلوک خالی و پایان بلوک خالی داشته باشند.
۳. با استفاده از داده ساختار جدید `freelist` دو تابع `alloc(size)` و `free(start,size)` را پیاده‌سازی کنید. همین‌طور توجه کنید که در صورتی که یک بلوک دقیقا قبل یا بعد از یک بلوک خالی به تازگی خالی شود یا کل فضای میان ۲ بلوک خالی باشد، باید نواحی خالی متصل به یکدیگر بپیوندند و نمی‌توانیم ۲ بلوک خالی همجوار داشته باشیم.
۴. عملیات‌های گذشته سامانه را با استفاده از `alloc` و `free` پیاده‌سازی کنید.
۵. دستور `viz` را به رابط کاربری اضافه کنید به طوری که فضاهای خالی را به شکل مرتب شده‌ای نمایش دهد.